E-Commerce RESTFUL API V1

StartFragment

E-Commerce RESTful API (V1) — Postman Collection

This collection provides a complete set of REST API requests for testing and interacting with an **E-Commerce backend (Version 1)**. It covers the core resources and typical workflows needed to manage an online store, including authentication, user management, product catalog, shopping cart, orders, reviews, brands, categories/subcategories, and discount coupons.

What's included

- **Authentication**: sign up and sign in to obtain access for protected operations.
- **Users**: create users, list/get users, update user data, delete users, and change passwords.
- **Categories & Subcategories**: CRUD operations for organizing the product catalog.
- **Brands**: CRUD operations for product branding.
- **Products**: create, list, get, update, and delete products.
- **Cart**: add items to cart, retrieve cart, update cart, and delete cart.
- **Orders**: create orders, list/get orders, update/cancel orders, and delete orders.
- **Reviews**: create, list/get, update, and delete product reviews.
- **Coupons**: create, list, update, and delete discount coupons.
- **Logout**: invalidate/end a user session.

Notes

- Requests are organized into folders by feature (e.g., `product`, `order`, `cart`, `user`, etc.) to make it easy to navigate and run related endpoints together.
- Many requests use a `https://ecommerce-backend-production-8996.up.railway.app` variable to support switching between environments (local/dev/staging) without editing individual request URLs.
- This collection uses the `https://ecommerce-backend-production-8996.up.railway.app` variable. Configure it in your Postman environment to match your deployment (e.g., local or production
EndFragment

---

category

POST addcategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories

Creates a new product category in the e-commerce API.

Endpoint

- **Method:** POST
- **URL:https://ecommerce-backend-production-8996.up.railway.app** `/api/v1/categories`

Note: This request currently uses `https://ecommerce-backend-production-8996.up.railway.app`

in the URL. The intended format is typically `https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories`. This description documents the intended endpoint; do not change the request URL unless you choose to update it separately.

Authentication

Send a JWT token in the header:

- `token:`

Example:

```
Plain Text

token: {{token}}
```

Body (multipart/form-data)

Required fields:

- `name` (string) — Category name (e.g., `electronics`)

Example cURL

```bash
curl --location --request POST "https://ecommerce-backend-production-8996.up.railway.app/api/v1
  --header "token: <JWT>"
  --form "name=electronics"
```

Expected Responses

Success

- **201 Created** (common)
  - Returns the created category object (typically includes fields like `_id` / `id`, `name`, `slug`, timestamps).

Common Errors

- **400 Bad Request**
  - Missing/invalid `name` or validation failure.
- **401 Unauthorized**
  - Missing/invalid token.
- **403 Forbidden**
  - Authenticated but not allowed (e.g., non-admin).
- **409 Conflict** (if implemented)
  - Category name already exists.
- **500 Internal Server Error**
  - Unexpected server error.

Notes

- Ensure `https://ecommerce-backend-production-8996.up.railway.app` is set in the active environment before sending the request.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body formdata

name	"electronics"
image	

GET getAllCategory

https://ecommerce-backend-production-8996.up.railway.app}/api/v1/categories

Purpose

Retrieve the list of all categories.

Method & URL

Suggestion: store the base host as an environment variable (e.g., `{{baseUrl}}`) and use:

```http
GET {{baseUrl}}/api/v1/categories
```

Authentication

None.

Query Parameters

None.

Example Response Schema (generic)

A successful response typically returns a JSON object containing a list of categories.

```json
{
  "results": 0,
  "data": [
    {
      "_id": "string",
      "name": "string",
      "slug": "string",
      "createdAt": "string (ISO 8601)",
      "updatedAt": "string (ISO 8601)"
    }
  ]
```

Common Status Codes

- **200 OK** — Categories returned successfully.
- **400 Bad Request** — Invalid request.
- **401 Unauthorized** / **403 Forbidden** — If auth is enabled on the API.
- **500 Internal Server Error** — Unexpected server error.

Notes

- This request currently targets a local server (`localhost`). Consider using an environment variable for the base URL so you can easily switch between local/staging/production.
- This endpoint is read-only and should not require a request body.

GET get category

https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/69877f4f902056165e7903af

Get Category

Retrieves a single category resource by its unique identifier.

Purpose

Use this endpoint to fetch the details of a specific category (for example: name, slug, metadata) given a category ID.

Endpoint

`GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/{categoryId}`

Path parameters

Name	Type	Required	Description
`categoryId`	string	Yes	The unique ID of the category to retrieve.

> In this saved request, the `categoryId` is currently set in the URL to `69877f4f902056165e7903af`.

Required variables

Variable	Required	Description
`baseURL`	Yes	Base URL for the API (protocol + host, and if applicable, base path).

Authentication

Not specified on this request. It **uses collection/workspace auth** if configured there.

Example request URL

```text
```

```
https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/69877f4f902056165e7
```

Typical responses

- **200 OK**: Category found and returned in the response body.
- **404 Not Found**: No category exists for the provided `categoryId`.
- **500 Internal Server Error**: Server-side error while retrieving the category.

PUT updatecategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/699cddc7e3edd84300eeafdb

Updates an existing category by ID.

Purpose

Use this request to rename (update) a category resource identified by `categoryId`.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/:categoryId`

- `baseURL` is a Postman variable resolved from the active environment **baseURL**.

Authentication

This API expects a `token` header.

Header

- `token: <Supabase Service Role API Key>`

> Keep this value secret. Store it in an environment/collection variable when possible.

Path parameter

- `categoryId` (string): The category identifier to update.

Example

- `699cddc7e3edd84300eeafdb`

Request body

Content-Type: `application/json`

Schema

```
 json

{
  "name": "string"
}
```

Example

```
 json

{
  "name": "sport and sport"
}
```

Expected responses

- **200 OK**: Category updated successfully. Response typically contains the updated category object.
- **400 Bad Request**: Validation error (e.g., missing/invalid `name`).
- **401 Unauthorized / 403 Forbidden**: Missing or invalid `token`.
- **404 Not Found**: No category exists for the provided `categoryId`.
- **500 Internal Server Error**: Unexpected server error.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```
 json

{
    "name":"sport and sport"
}
```

DELETE deletecategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/699dcc66a8072828689942fa

Delete Category

Deletes a single category by its ID.

> **Note**: This request currently uses a **hardcoded example ID** in the path (`699dcc66a8072828689942fa`). In practice you should replace it with a variable like `{{categoryId}}` or set the correct ID directly in the URL.

Authorization

This endpoint requires an authentication token passed via the `token` header:

```http
token: <your-token>
```

Required variables

- `https://ecommerce-backend-production-8996.up.railway.app` — Base URL of the API (provided by the active environment named **baseURL**).

Path parameter

- `categoryId` *(string, required)* — The ID of the category to delete.

Current request example:

```Plain Text
DELETE https://ecommerce-backend-production-8996.up.railway.app/api/v1/categories/699dcc66a807
```

Expected responses

- **200 OK** or **204 No Content** — Category deleted successfully.
- **401 Unauthorized** — Missing/invalid `token`.
- **403 Forbidden** — `token` is valid but not allowed to delete categories.
- **404 Not Found** — Category ID does not exist.
- **500 Internal Server Error** — Unexpected server-side error.

---

cURL example

```bash
curl --request DELETE "https://ecommerce-backend-production-8996.up.railway.app/api/v1/categor
  --header "token: <your-token>"
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

cart

POST addtocart

https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart

Purpose

Add a product to the authenticated user's cart.

Endpoint

- **Method:** `POST`
- **URL:** `{baseURL}}/api/v1/cart`

Authentication

This request requires a token header.

```http
token: {{token}}
```

- In this request, the `token` header is currently set to a Supabase Service Role API key.

Environment / Variables

- `https://ecommerce-backend-production-8996.up.railway.app` (from the active environment named **baseURL**)

baseURL)

Request Body

- **Content-Type:** `application/json`

Schema

```json
{
  "productId": "string" // Product ID to add to the cart
}
```

Example

```json
{
  "productId": "699cbf3e933c6d1eaacb6262"
}
```

Responses (generic)

Success

- `200 OK` or `201 Created`

```json
{
            "message": "Added to cart",
            "data": {
              "cartId": "string",
              "items": [
                {
                  "productId": "string",
                  "quantity": 1
                }
              ]
            }
```

Error

- `400 Bad Request` (missing/invalid `productId`)
- `401 Unauthorized` / `403 Forbidden` (missing/invalid token)
- `404 Not Found` (product does not exist)
- `409 Conflict` (product already in cart, depending on implementation)

```
{
  "message": "Error message"
}
```

Notes

- Keep the request body as-is; only `productId` is required here.
- Fix the extra `}` in the URL later when updating request configuration.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTU0NTE5fQ.27YqljWGq-8QlRCZnm5r8eis6Dp4I8SPMCZzWawKCUs

Body raw (json)

```
{
    "productId":"699cbf3e933c6d1eaacb6262"
}
```

GET get cart

https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/699dd94ff386c2f9d58d02ba

Get Cart by ID

Retrieves a single cart resource by its unique identifier.

Purpose

Use this endpoint to fetch the current state of a specific cart (e.g., items, quantities, totals) by cart ID.

Base URL / Environment

This request uses the environment variable `https://ecommerce-backend-production-8996.up.railway.app`

This request uses the environment variable `https://ecommerce-backend-production-8996.up.railway.app/`

Authentication

This endpoint requires an auth token.

This request is currently sending a `token` header:

```http
token: {{token}}
```

- Provide a valid token value (often a JWT) for an authorized user.
- Recommended practice: store the token in an environment/collection variable like `{{token}}` instead of hard-coding it.

Path parameter

`GET /api/v1/cart/{cartId}`

Parameter	In	Type	Required	Description
`cartId`	path	string	Yes	The unique identifier of the cart to retrieve.

Example `cartId` in this request:

- `699dd94ff386c2f9d58d02ba`

Example request

```http
GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/699dd94ff386c2f9d58d02
token: {{token}}
```

Example success response (200)

```json
{
  "status": "success",
  "data": {
    "id": "699dd94ff386c2f9d58d02ba"
```

```
    "id": "699dd941130bc217d30d02ba",
    "items": [
      {
        "productId": "<product-id>",
        "quantity": 1,
        "price": 0
      }
    ]
```

> Response fields vary by implementation. Use the example above as a representative shape.

Common errors

- **400 Bad Request**: Invalid `cartId` format.
- **401 Unauthorized**: Missing/invalid token.
- **403 Forbidden**: Token is valid but user does not have permission to access this cart.
- **404 Not Found**: No cart exists for the given `cartId`.
- **500 Internal Server Error**: Unexpected server error.

Troubleshooting tips

- If you see errors like **ENOTFOUND**, **Invalid URL**, or DNS/host issues, verify `https://ecommerce-backend-production-8996.up.railway.app` is correct and remove the extra `}` in the configured request URL.
- If you get **401**, ensure the `token` header is present and not expired.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

PUT updatecart

https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/699cbf3e933c6d1eaacb6262

Update cart item quantity

Updates the quantity for a specific cart (or cart item entry) identified by `cartId`.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/:cartId`

Authentication

This endpoint requires a valid JWT.

Header used in this request

```http
token: <JWT>
```

> Note: The token is currently hardcoded in the request header. Recommended: store it as a variable (e.g., `{{token}}`) in your environment or collection.

Path parameter

Param	Type	Required	Description
`cartId`	string	Yes	The cart identifier to update (e.g., `699cbf3e933c6d1eaacb6262`).

Request body

Content-Type: `application/json`

Schema:

```json
{
  "quantity": 3
}
```

Field	Type	Required	Description
`quantity`	number (integer)	Yes	New quantity for the cart item. Typically must be `>= 1`.

Example request

```http
PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/699cbf3e933c6d1eaacb6
token: {{token}}
Content-Type: application/json
{"quantity":3}
```

Expected responses

> Exact response bodies may vary by implementation.

- **200 OK** — Quantity updated successfully; returns the updated cart/cart-item representation.
- **400 Bad Request** — Validation error (e.g., missing/invalid `quantity`).
- **401 Unauthorized** — Missing or invalid JWT token.
- **403 Forbidden** — Authenticated but not allowed to update this cart.
- **404 Not Found** — `cartId` does not exist.
- **500 Internal Server Error** — Unexpected server error.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```
{
    "quantity":3
}
```

DELETE deletecart

https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/699cbf3e933c6d1eaacb6262

Purpose

Deletes a cart by ID.

Method: `DELETE`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/:cartId`

Authorization

This endpoint requires an auth token sent in a request header:

```http
```

```
token: <JWT-or-token-value>
```

Notes:

- In this request, the header key is `token` (not `Authorization`).
- Typically you obtain this token from your sign-in/login endpoint.

Path parameter

- `cartId` *(string, required)*: The cart identifier to delete.

Example:

```http
DELETE https://ecommerce-backend-production-8996.up.railway.app/api/v1/cart/699cbf3e933c6d1eaa
```

Environment variable

- `https://ecommerce-backend-production-8996.up.railway.app`: The API base URL (from the active environment named **baseURL**).

Expected responses

Because implementations vary, use the examples below as a guide and confirm with the actual API response.

Success

- `200 OK` or `204 No Content` — cart deleted successfully.

Common errors

- `400 Bad Request` — invalid `cartId` format.
- `401 Unauthorized` — missing/invalid/expired `token` header.
- `403 Forbidden` — token is valid but user lacks permission.
- `404 Not Found` — cart with `cartId` not found.
- `500 Internal Server Error` — unexpected server error.

Notes / Known issue in current request URL

The currently configured request URL appears to contain an extra closing brace: `https://ecommerce-backend-production-8996.up.railway.app}` (three `}` characters).
This will usually result in an invalid URL.

> Per your instruction, this documentation notes the issue but does **not** modify the request URL.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

review

POST addreview

https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews

Add Review

Create a new review for a product.

Endpoint

`POST https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews`

> **Note (current request URL typo):** the request URL currently appears as `"https://ecommerce-backend-production-8996.up.railway.app}/api/v1/reviews"` (there is an extra `}` after `baseURL`). It should be `https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews`.

Authentication

Requires an authenticated user.

This request currently uses a custom header named `token` that contains a JWT.

Required headers

Header	Type	Required	Description
`token`	string	Yes	JWT access token for the authenticated user.
`Content-Type`	string	Yes*	Use `application/json` when sending a JSON body. (*Postman sets this automatically for raw JSON in many

			cases.)

Request body (JSON)

Field	Type	Required	Description
`text`	string	Yes	Review text/content.
`productId`	string	Yes	The product identifier being reviewed.
`rate`	number	Yes	Rating value (e.g., 3.6).

Example request body

```json
{
  "text": "good product",
  "productId": "6995faf7a06db4684ab790d1",
  "rate": 3.6
}
```

Example success response

> Response shape may vary by API implementation.

```json
{
  "message": "Review created successfully",
  "data": {
    "_id": "<reviewId>",
    "text": "good product",
    "productId": "6995faf7a06db4684ab790d1",
    "rate": 3.6,
    "createdAt": "2026-01-01T00:00:00.000Z"
  }
}
```

Common errors

Status	When it happens	Typical causes
400	Validation error	Missing/invalid `text`, `productId`, or `rate`.
401	Unauthorized	Missing/expired/invalid `token` header.
403	Forbidden	Authenticated but not allowed to create a review (role/permissions).
404	Not Found	`productId` does not exist.
409	Conflict	Review already exists (if API prevents duplicates).
500	Server error	Unexpected server-side failure.

Variables & notes

- `https://ecommerce-backend-production-8996.up.railway.app` is resolved from the active environment (currently named **baseURL**).
- Keep tokens out of the request description for security; use environment/collection variables where possible.

HEADERS

token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmoZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```json
{
    "text":"good product ccccc",
    "productId":"6995faf7a06db4684ab790d1",
    "rate":3.6
}
```

GET getallreviews

https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews

Get Reviews

Retrieves a list of reviews from the API.

- **Method:** `GET`
- **Endpoint:** `https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews`

> Note: the current saved request URL appears as `https://ecommerce-backend-production-8996.up.railway.app}/api/v1/reviews` (extra `}`), which may cause an invalid URL when sending. The intended pattern is shown above.

Authentication

This endpoint requires a valid JWT token.

Current request header (as saved)

This request currently sends the token using a custom header named `token`:

```http
token: <jwt>
```

Common alternative (if your API expects it)

Some APIs expect the token in the standard `Authorization` header:

```http
Authorization: Bearer <jwt>
```

Use whichever header format your backend expects. (This documentation does not change the saved request.)

Base URL (`https://ecommerce-backend-production-8996.up.railway.app`) environment variable

This request uses the `https://ecommerce-backend-production-8996.up.railway.app` variable.

- Define `baseURL` in your active environment (example: the `baseURL` environment).
- Expected format:

```text
```

```
https://your-domain.com
```

- Avoid trailing braces or typos (for example, do **not** include an extra `}`), and typically avoid a trailing slash unless your collection standard requires it.

Query Parameters

No query parameters are documented in this request.

If implemented by your API, this endpoint may support **pagination and filtering** (for example: `page`, `limit`, `sort`, `keyword`, `rating`, `productId`, etc.). Refer to your backend implementation for the exact parameter names.

Example response schema

Typical success response is expected to be JSON.

```json
[
  {
    "_id": "string",
    "title": "string",
    "ratings": 0,
    "comment": "string",
    "user": {
      "_id": "string",
      "name": "string"
    },
    "product": {
```

> Field names may vary depending on your backend model (e.g., `rating` vs `ratings`, whether `user` / `product` are IDs vs populated objects).

Basic expectations (when running)

When you send this request with a valid token:

- Should return **HTTP 200 OK**.
- Response body should be **JSON**.
- Response should be an **array** of review objects (may be empty `[]`).

If the token is missing/invalid/expired:

- Common outcomes: **401 Unauthorized** or **403 Forbidden** (depending on backend behavior).

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2Ii wicm9sZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

POST getsinglereview

https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews/id

Purpose

Create a new **review** for a resource (commonly a product) in the e-commerce API.

Endpoint

`POST https://ecommerce-backend-production-8996.up.railway.app}/api/v1/reviews/id`

> Note: The URL currently includes an extra `}` after `https://ecommerce-backend-production-8996.up.railway.app`. This documentation reflects the request exactly and **does not change** it.

Authentication

This endpoint typically requires a Bearer token.

Send:

```http
Authorization: Bearer {{token}}
```

Request data (assumptions)

This request is documented as **creating** a review.

- **Path / identifier**: The trailing `id` in the path is assumed to represent the target resource identifier (for example `productId`).
- **Body**: A JSON body is typically required even though this Postman request currently has **no body** configured.

Common body fields (example)

Assuming the API follows standard review patterns, the body may include:

Field	Type	Required	Notes
`rating`	number	Yes	Usually 1–5

`title`	string	No	Short summary
`comment`	string	Yes	The review text

Example request

http

```http
POST https://ecommerce-backend-production-8996.up.railway.app}/api/v1/reviews/id
Authorization: Bearer {{token}}
Content-Type: application/json
{
  "rating": 5,
  "title": "Great product",
  "comment": "Fast shipping and excellent quality."
}
```

Example response (201 Created)

json

```json
{
  "status": "success",
  "data": {
    "review": {
      "_id": "65f0c1b2a1b2c3d4e5f67890",
      "rating": 5,
      "title": "Great product",
      "comment": "Fast shipping and excellent quality.",
      "createdAt": "2026-03-01T12:34:56.000Z"
    }
  }
```

Common status codes

- **201 Created**: Review created successfully.
- **400 Bad Request**: Missing/invalid fields (e.g., rating out of range).
- **401 Unauthorized**: Missing/invalid Bearer token.
- **403 Forbidden**: Authenticated but not allowed (e.g., not permitted to review).
- **404 Not Found**: Target `id` does not exist.
- **409 Conflict**: Duplicate review (if only one review per user is allowed).
- **500 Internal Server Error**: Unexpected server error.

PUT updatereview

https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews/699dd7dcea4a82568981b853

Update review

Updates the text/content of an existing review by its ID.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews/:reviewId`

Authentication

This endpoint requires an authenticated user.

Send a JWT in the request header:

```http
token: {{jwt}}
```

> Note: This API uses a custom `token` header (not `Authorization: Bearer ...`).

Path parameters

Name	Type	Required	Description
`reviewId`	string (ObjectId)	Yes	The ID of the review to update.

Example:
`/api/v1/reviews/699dd7dcea4a82568981b853`

Request body (JSON)

`Content-Type: application/json`

Schema:

```json
{
  "text": "string"
}
```

Field details:

Field	Type	Required	Description
`text`	string	Yes	Updated review text.

Example:

```json
{
  "text": "thats is very good review"
}
```

Responses

Possible outcomes (exact body may vary by implementation):

- **200 OK** — Review updated successfully; returns the updated review object.
- **400 Bad Request** — Invalid payload (e.g., missing/invalid `text`).
- **401 Unauthorized** — Missing/invalid JWT in the `token` header.
- **403 Forbidden** — Authenticated but not allowed to update this review.
- **404 Not Found** — No review found for the given `reviewId`.
- **500 Internal Server Error** — Unexpected server error.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```json
{
    "text":"thats is very good review"
}
```

DELETE delete review

https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews/699dd7dcea4a82568981b853

Delete Review

Deletes a single review by its id.

Purpose

Use this endpoint to permanently remove an existing review resource from the system.

Authentication

Requires an authenticated user with sufficient permissions.

Send the token in the request header:

```http
token: <JWT>
```

> Note: In this collection, the auth header name is `token` (not `Authorization`).

Path parameters

Param	Type	Required	Description
`id`	string (MongoDB ObjectId)	Yes	The review id to delete.

Example request

```http
DELETE https://ecommerce-backend-production-8996.up.railway.app/api/v1/reviews/699dd7dcea4a825
token: {{token}}
```

Example response (success)

Typical successful responses may be either **200 OK** (with a JSON body) or **204 No Content** (empty body), depending on the API implementation.

```json
{
  "status": "success",
  "message": "Review deleted"
}
```

Common status codes

- **200 OK** — Review deleted (may include a confirmation payload)
- **204 No Content** — Review deleted (no response body)
- **400 Bad Request** — Invalid `id` format
- **401 Unauthorized** — Missing/invalid token
- **403 Forbidden** — Authenticated but not allowed to delete this review

- **404 Not Found** — Review not found
- **500 Internal Server Error** — Unexpected server error

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

order

POST addorder

https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders

Purpose

Create a new order for the authenticated user.

Endpoint

`POST https://ecommerce-backend-production-8996.up.railway.app:3000/api/v1/orders`

> **Note:** This request uses the `https://ecommerce-backend-production-8996.up.railway.app` environment variable. Ensure the active environment defines `baseURL`.

Authentication

This endpoint requires an auth token sent in a custom header:

```http
token: <JWT or access token>
```

If the token is missing/invalid/expired, you should receive `401` or `403`.

Required environment variables

- `baseURL` — Base host/protocol for the API (for example: `http://localhost` or `https://api.example.com`).

Request body (JSON)

Content-Type: `application/json`

Schema

```json
{
  "shippingAddress": {
    "details": "string",
    "city": "string",
    "phone": "string"
  },
  "paymentMethod": "cash" | "card" | "<other supported method>"
}
```

Field descriptions

- `shippingAddress` *(object, required)*: Shipping destination information.
  - `details` *(string, required)*: Street/address details (e.g., building, street, apartment).
  - `city` *(string, required)*: City name.
  - `phone` *(string, required)*: Contact phone number.
- `paymentMethod` *(string, required)*: Payment method for the order (example: `cash`).

Example request

```http
POST https://ecommerce-backend-production-8996.up.railway.app:3000/api/v1/orders
token: {{token}}
Content-Type: application/json
{
  "shippingAddress": {
    "details": "Street 10",
    "city": "Cairo",
    "phone": "01000000000"
  },
  "paymentMethod": "cash"
}
```

Expected responses

`201 Created`

Order created successfully.

Typical response contains the created order object (and/or metadata), depending on your API implementation.

`400 Bad Request`

Validation error or missing/invalid fields in the request body.

`401 Unauthorized` / `403 Forbidden`

Missing token, invalid token, or insufficient permissions.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcyMDM5NDUwfQ.x9P7pRW6B66fMoEgpWNCOafINvd1g5cSb8HOS9Mh2UY

Body raw (json)

```
 json

{
  "shippingAddress": {
    "details": "Street 10",
    "city": "Cairo",
    "phone": "01000000000"
  },
  "paymentMethod": "cash"

}
```

GET getorders

https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders

Purpose

Retrieves the list of orders.

Endpoint: `GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders`

Authentication

This endpoint requires an access token sent in a custom header:

- Header: `token:`

Notes:

- If the token is missing, expired, or invalid, the API typically returns **401 Unauthorized**.
- If the token is valid but the user is not allowed to access orders, the API typically returns **403 Forbidden**.

Base URL variable (`https://ecommerce-backend-production-8996.up.railway.app`)

The request URL uses the Postman variable `https://ecommerce-backend-production-8996.up.railway.app`.

- Ensure the active environment (or collection/global variable) defines `baseURL`.
- Example value: `https://api.your-domain.com` (no trailing slash preferred).

Expected success response (200)

A successful response returns **200 OK** and a JSON payload containing a list of orders.

Generic shape (example):

```json
{
  "results": 0,
  "data": [
    {
      "_id": "string",
      "user": "string|object",
      "items": [
        {
          "product": "string|object",
          "quantity": 0,
          "price": 0
```

(Exact fields may vary by implementation.)

Common error responses

- **401 Unauthorized**: Token header missing/invalid/expired.
- **403 Forbidden**: Authenticated but not permitted to view orders.

---

cURL example

```bash
curl --location "https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders" \
  --header "token: <YOUR_JWT_TOKEN>"
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcyMDM5NDUwfQ.x9P7pRW6B66fMoEgpWNCOafINvd1g5cSb8HOS9Mh2UY

GET getorder

https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699cc803ad9d5bfff76b92a7

Get Order by ID

Purpose

Retrieve a single order record by its unique identifier. Common use cases include:

- Viewing order details in an admin/dashboard UI
- Validating an order exists before performing follow-up actions (refund, status update, etc.)

Endpoint

`GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/:orderId`

- **Method:** `GET`
- **Current request URL in this collection:**

  Plain Text

  ```
  https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699cc803ad9d5bfff76
  ```

Variables

- `https://ecommerce-backend-production-8996.up.railway.app`
  - **Scope:** Environment variable (active environment: `baseURL`)
  - **Description:** Base URL of the API server (for example `http://localhost:3000` or `https://api.yourdomain.com`).

Path Parameters

- `orderId` (string)

- **Description:** The unique identifier of the order to retrieve.
- **Note:** In this request, `orderId` is currently **hardcoded** in the URL (`699cc803ad9d5bfff76b92a7`).

- **Recommendation:** Replace the hardcoded value with a variable for easier reuse, for example:
  - URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/{{orderId}}`
  - and store `orderId` in an environment/collection variable.

Required Headers

- `token:`
  - **Required:** Yes
  - **Description:** Authentication token used by the API to authorize the request.
  - **Notes:**
    - This request uses a custom `token` header (not the standard `Authorization: Bearer ...`).
    - Keep tokens out of shared/public documentation and prefer storing them as variables.

Example Request

cURL

```bash
curl --request GET "https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699
  --header "token: {{token}}"
```

Postman

- Method: `GET`
- URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699cc803ad9d5bfff76b92a7`
- Header: `token:`

Expected Responses

> Response shapes can vary by implementation. Schemas below are **brief** and represent typical patterns.

`200 OK`

Order was found and returned.

Body (typical schema)

```json
```

```
{
  "status": "success",
  "data": {

    "order": {
      "_id": "string",
      "user": "string|object",
      "items": [
        {
          "product": "string|object",
          "quantity": "number",
```

`401 Unauthorized`

Missing/invalid token.

Body (typical schema)

```
 json

{
  "status": "fail",
  "message": "string"
}
```

`403 Forbidden`

Token is valid but the user does not have permission to access this order (role/ownership rules).

Body (typical schema)

```
 json

{
  "status": "fail",
  "message": "string"
}
```

`404 Not Found`

No order exists for the provided `orderId`.

Body (typical schema)

```
 json

{
  "status": "fail",
  "message": "string"
}
```

`500 Internal Server Error`

Unexpected server-side error.

Body (typical schema)

```json
{
  "status": "error",
  "message": "string"
}
```

Authentication Notes

- This endpoint requires authentication via the `token` header.
- Ensure your token is:
  - not expired
  - associated with a user that has permission to read the requested order
- Best practice: store the token in a variable (for example `{{token}}`) rather than pasting raw JWTs into headers.

Environment Setup Notes

1. Select the **`baseURL`** environment.
2. Set `baseURL` to the correct API host (for example `http://localhost:3000`).
3. (Recommended) Add variables:
   a. `token` (JWT)
   b. `orderId` (an existing order ID)

Related

- Collection: e-commerce
- Folder: Folder

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcyMDM5NDUwfQ.x9P7pRW6B66fMoEgpWNCOafINvd1g5cSb8HOS9Mh2UY

PUT updateorder

https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699de2c1de095762390035e8

Purpose

Update an existing order's payment/fulfillment status (e.g., mark an order as paid).

Endpoint & Method

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/:orderId`

Required variables

- `https://ecommerce-backend-production-8996.up.railway.app` (from the active environment **baseURL**): API base URL, e.g. or `https:https://ecommerce-backend-production-8996.up.railway.app//api.your-domain.com`

Path parameter

- `orderId` (string / ObjectId): The unique identifier of the order to update.
  - In this request tab it is currently set to: `699de2c1de095762390035e8`

Required headers

- `token:`
  - Must be a valid JWT for a user allowed to update orders (often an admin or the order owner, depending on your API rules).

Request body

Content-Type: `application/json` (recommended)

Schema

```json
{
  "status": "string"
}
```

Allowed values

- `status` : `"paid"`

Example request

cURL

```bash
```

```bash
curl --request PUT "https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699
  --header "token: <jwt>" \
  --header "Content-Type: application/json" \
  --data '{"status":"paid"}'
```

Raw JSON body

```json
{
  "status": "paid"
}
```

Possible responses

- **200 OK**: Order updated successfully (response typically includes the updated order or an acknowledgement message).
- **400 Bad Request**: Invalid/missing body fields (e.g., missing `status` or unsupported value).
- **401 Unauthorized**: Missing/invalid `token`.
- **403 Forbidden**: Authenticated but not allowed to update this order.
- **404 Not Found**: No order exists for the given `orderId`.
- **500 Internal Server Error**: Unexpected server error.

Notes

- Prefer parameterizing the order id for reusability, e.g. `PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/{{orderId}}` with `orderId` stored as an environment/collection variable.
- Ensure the `token` header value is current (JWTs can expire).
- If your server enforces content type, add `Content-Type: application/json`.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTU0NTE5fQ.27YqljWGq-8QIRCZnm5r8eis6Dp4I8SPMCZzWawKCUs

Body raw (json)

```json
```

```
{
    "status":"paid"
}
```

PATCH cancel

https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699de2c1de095762390035e8

PATCH Update Order (by ID)

Updates an existing order using its unique identifier.

Endpoint

`PATCH https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/:orderId`

Authentication

This request uses a JWT passed in a request header named `token`.

Header

Plain Text

```
token: <jwt>
```

> Tip: You can store the token in an environment/collection variable and reference it as `{{token}}`.

Path parameters

Name	Type	Required	Description
`orderId`	string	yes	The order ID to update (example: `699de2c1de095762390035e8`).

Request body

No request body is currently configured for this request.

If your API expects a payload for partial updates, add a JSON body (example shape depends on your backend), e.g.

ison

```
{
  "status": "shipped"
}
```

Expected responses

Common outcomes you might see:

- **200 OK / 204 No Content** — Order updated successfully.
- **400 Bad Request** — Invalid `orderId` format or invalid update payload.
- **401 Unauthorized** — Missing/invalid JWT in the `token` header.
- **403 Forbidden** — Authenticated but not allowed to update this order.
- **404 Not Found** — Order with `orderId` does not exist.
- **500 Internal Server Error** — Unexpected server error.

Usage examples

1) Update a specific order (current request as saved)

Plain Text

```
PATCH https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699de2c1de0957623
Headers:
  token: <jwt>
```

2) Parameterize the order ID
Update the URL to use a variable:

Plain Text

```
PATCH https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/{{orderId}}
```

Then set `orderId` in your environment/collection variables before sending.

3) cURL (illustrative)

bash

```
curl --request PATCH "https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/69
  --header "token: <jwt>"
```

Notes

- Use `PATCH` for partial updates (send only the fields you want to change).
- If your API requires `Content-Type: application/json`, add it when you add a JSON body.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTU0NTE5fQ.27YqljWGq-8QlRCZnm5r8eis6Dp4I8SPMCZzWawKCUs

PARAMS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiYmVzaG95IG1vZmllZCIsImVtYWlsIjoiYmVzaG95bW9maWVkQGdtYWlsLmNvbSIsImlkIjoiNjk5NWY5ZTBhMDZkYjQ2ODRhYjc5MGM4IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxNDM2NTU1fQ.gBcvqyrgUZNeay4PQOzTY0YelEi2hI_Qp5RzbbUCIDU

DELETE deleteorder

https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699cc803ad9d5bfff76b92b4

Delete Order

Deletes an existing order by ID.

Method: `DELETE`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/:orderId`

Authentication

This endpoint requires an admin/service token.

- Header: `token: {{token}}`

> Keep tokens out of hard-coded headers when possible—store them in an environment/collection variable and mark the value as *secret*.

Path parameter

Param	Type	Required	Description
`orderId`	string	yes	The unique ID of the order to delete.

Example usage

Replace `:orderId` with the target order ID:

http

```http
DELETE https://ecommerce-backend-production-8996.up.railway.app/api/v1/orders/699cc803ad9d5bff
token: {{token}}
```

Expected responses

Common outcomes (actual payload may vary by implementation):

- **200 OK / 204 No Content** — order deleted successfully
- **400 Bad Request** — invalid `orderId`
- **401 Unauthorized / 403 Forbidden** — missing/invalid token or insufficient permissions
- **404 Not Found** — order does not exist
- **500 Internal Server Error** — unexpected server error

Notes

- `https://ecommerce-backend-production-8996.up.railway.app` is resolved from the active environment (**baseURL**). Make sure it points to the correct server (dev/staging/prod).
- Use the minimum required privileges for the token (avoid using high-privilege service-role tokens outside trusted server contexts).

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTU0NTE5fQ.27YqljWGq-8QIRCZnm5r8eis6Dp4I8SPMCZzWawKCUs

subcategory

POST addsubcategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories

Create Subcategory

Creates a new subcategory under an existing category.

Method: `POST`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories`

Authentication

This endpoint requires an auth token in a request header:

```http
token: <YOUR_TOKEN>
```

> In this request, the header key is literally `token` (not `Authorization`).

Required environment variables

This request uses the following variable:

- `baseURL` — the API base URL (e.g., `http://localhost:3000`)

Request body (JSON)

Content-Type: `application/json`

Schema

Field	Type	Required	Description
`name`	string	Yes	Subcategory name.
`category`	string	Yes	Parent category ID (typically a MongoDB ObjectId).
`createdBy`	string	Yes	User ID of the creator (typically a MongoDB ObjectId).

Example

```json
{
  "name": "mobile and phone",
  "category": "699b2f50c20ede524eb11a9b",
  "createdBy": "6995f9e0a06db4684ab790c8"
}
```

Example success response (generic)

> Exact fields may vary by implementation.

Status: `201 Created`

```json
{
  "message": "Subcategory created successfully",
  "data": {
    "_id": "<subcategory_id>",
    "name": "mobile and phone",
    "category": "<category_id>",
    "createdBy": "<user_id>",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-01-01T00:00:00.000Z"
  }
}
```

Common error responses

400 Bad Request — validation error (missing/invalid fields)

```json
{
  "message": "Validation error",
  "errors": [
    {
      "field": "name",
      "message": "name is required"
    }
  ]
}
```

401 Unauthorized — missing/invalid `token`

```json
{
  "message": "Unauthorized"
}
```

404 Not Found — referenced `category` (or related resource) not found

```json
{
  "message": "Category not found"
}
```

409 Conflict — duplicate subcategory name within a category (if enforced)

```
 json

{
  "message": "Subcategory already exists"
}
```

500 Internal Server Error — unexpected server error

```
 json

{
  "message": "Internal server error"
}
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```
 json

{
    "name":"mobile and phone",
    "category":"699b2f50c20ede524eb11a9b",
    "createdBy":"6995f9e0a06db4684ab790c8"
}
```

GET getAllSubCategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories

GET `https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories`

Purpose

Purpose

Retrieve a list of subcategories.

- **Base URL variable**: This request uses `https://ecommerce-backend-production-8996.up.railway.app` (configure it in your active environment).
- **Returns**: A paginated or non-paginated list (as implemented by the API) of subcategory resources.

Authentication

As configured / inherits collection auth.

This request does not define request-level auth in this request. It will use whatever authentication is configured at the folder/collection level (or none, depending on your setup).

Query Parameters

None currently.

Example response shape (generic)

```json
{
  "results": 0,
  "data": [
    {
      "id": "string",
      "name": "string",
      "slug": "string",
      "category": {
        "id": "string",
        "name": "string"
      },
```

> Note: Field names and wrapping keys can vary by implementation. Use this as a generic shape for “list subcategories”.

Common status codes

- **200 OK**: Successfully returned the list.
- **400 Bad Request**: Invalid query parameters (if added later).
- **401 Unauthorized**: Missing/invalid authentication (if required by the API).
- **403 Forbidden**: Authenticated but not allowed to access subcategories.
- **500 Internal Server Error**: Unexpected server error.

GET get subcategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/:id

Get Subcategory by ID

Fetch a single **subcategory** resource by its unique identifier.

Endpoint

`GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/:id`

Required variables

- `https://ecommerce-backend-production-8996.up.railway.app` *(required)*: Base URL of the API (for example: `https://api.yourdomain.com`).

Path parameters

Param	Type	Required	Description
`id`	string	yes	The subcategory ID to retrieve.

Example usage

Request

```http
GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/64f1c2ab12cd
```

> Tip: You can also set the path variable in Postman by filling `id` in the **Path Variables** table.

Typical responses

- **200 OK** — Subcategory returned successfully (response body contains the subcategory data).
- **400 Bad Request** — Invalid `id` format.
- **404 Not Found** — No subcategory exists for the provided `id`.
- **500 Internal Server Error** — Unexpected server error.

PATH VARIABLES

id

PUT updatecategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/699dd38bd0761e12a2905a87

Update Subcategory

Updates an existing subcategory's details by ID.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/:subcategoryId`

Authentication (required)

Send a valid JWT in the `token` header:

```http
token: <JWT>
```

Path parameter

Name	Type	Required	Description
`subcategoryId`	string	Yes	The subcategory identifier to update.

Request body

Content-Type: `application/json`

Schema (JSON):

```json
{
  "name": "string"
}
```

Example:

```json
{
  "name": "klmvl;fd ccvm,"
}
```

Responses (generic)

Success

- **200 OK** (or **204 No Content**)
  - Returns the updated subcategory object (implementation-dependent).

Errors

- **400 Bad Request** — validation error (e.g., missing/invalid `name`).
- **401 Unauthorized** — missing/invalid JWT in `token` header.
- **403 Forbidden** — authenticated but not allowed to update.
- **404 Not Found** — `subcategoryId` does not exist.
- **500 Internal Server Error** — unexpected server error.

Notes

- `https://ecommerce-backend-production-8996.up.railway.app` is an environment variable (active environment: `baseURL`) that should point to your API host, e.g. `http://localhost:3000` or `https://api.yourdomain.com`.
- This request currently uses a concrete ID in the URL; you can replace it with `:subcategoryId` (or a variable like `{{subcategoryId}}`) for reuse when you choose to update the request later.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```
json

{
    "name":"klmvl;fd ccvm,"
}
```

DELETE deletecategory

https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/699dd38bd0761e12a2905a87

Delete Subcategory

Deletes a subcategory by its unique ID.

Endpoint

`DELETE https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcategories/:subcategoryId`

Authentication

This endpoint requires a valid service token sent in the `token` header.

Plain Text

```
token: {{token}}
```

> In this request, the `token` header is currently set directly. Consider storing it in an environment/collection variable (e.g., `{{token}}`) to avoid hard-coding secrets.

Path parameter

- `subcategoryId` (string, required): The subcategory ID to delete.

Example ID (as currently used):

- `699dd38bd0761e12a2905a87`

Required variables

- `baseURL`: Base URL for the API (active environment: `baseURL`).

Expected responses

Common responses you may see:

- **200 OK** / **204 No Content** — subcategory deleted successfully (depending on API implementation).
- **400 Bad Request** — invalid ID format or request validation failed.
- **401 Unauthorized** / **403 Forbidden** — missing/invalid token or insufficient permissions.
- **404 Not Found** — subcategory ID does not exist.
- **500 Internal Server Error** — unexpected server error.

Example cURL

bash

```bash
curl --request DELETE "https://ecommerce-backend-production-8996.up.railway.app/api/v1/subcateg
  --header "token: <YOUR_SERVICE_TOKEN>"
```

HEADERS

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

brands

POST add brand

https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands

Add Brand

Creates a new **brand** resource.

Method: `POST`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands`

Authentication

Send your JWT in the request header:

```http
token: {{token}}
```

> The API expects a header named `token` (not `Authorization`).

Body (multipart/form-data)

Send as **form-data** (multipart):

Field	Type	Required	Description
`name`	text	Yes	Brand name.
`logo`	file	No	Brand logo image file (if supported by the API).

Example request

cURL

```bash
curl --request POST "https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands" \
  --header "token: <JWT>" \
  --form "name=yo0yo" \
  --form "logo=@/path/to/logo.png"
```

Postman form-data

- `name` : `yo0yo`
- `logo` : *(choose a file)*

Possible responses

> Exact response schema may vary by implementation.

201 Created

Brand created successfully.

400 Bad Request

Validation error (for example: missing/empty `name`, invalid file type/size for `logo`).

401 Unauthorized / 403 Forbidden

Missing, invalid, or expired `token`, or insufficient permissions.

409 Conflict

Brand already exists (for example: duplicate name).

500 Internal Server Error

Unexpected server error.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body formdata

name	"yo0yo"
logo	

GET get brands

https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands

Purpose

Fetches the list of available **brands** in the E-Commerce API.

Endpoint: `GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands`

Authentication

This request does **not** define auth at the request level.

- If your API requires auth, configure it at the **collection/folder** level (recommended), or set it here.
- Common patterns:
  - Bearer token: `Authorization: Bearer {{token}}`
  - API key header: `x-api-key: {{apiKey}}`

Variables

This request uses the following variable:

- `https://ecommerce-backend-production-8996.up.railway.app` — Base URL of the API (from the active environment **baseURL**).

Example value:

text

```
https://api.example.com
```

Example Response (200 OK)

Example response shape (may vary by implementation):

json

```
{
  "results": 2,
  "data": [
    {
      "_id": "65f0b2c9e1a2b3c4d5e6f789",
      "name": "Nike",
      "slug": "nike",
      "image": "https://cdn.example.com/brands/nike.png",
      "createdAt": "2026-01-10T12:34:56.789Z",
      "updatedAt": "2026-01-10T12:34:56.789Z"
```


Common Errors

- **400 Bad Request** — Invalid query parameters (if your API supports filtering/pagination).
- **401 Unauthorized** — Missing/invalid credentials (when auth is required by the API).
- **403 Forbidden** — Authenticated but not allowed to access brands.
- **404 Not Found** — Incorrect route or base URL (check `https://ecommerce-backend-production-8996.up.railway.app`).
- **500 Internal Server Error** — Server-side error.

Notes

- No request body is sent for this `GET` request.
- If you add query params (e.g., pagination), document them in the Params tab and update this description accordingly.

GET get brand

https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/:id

Get brand by ID

Retrieves a single brand resource by its unique identifier.

Method: `GET`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/:id`

Path variable

Name	Type	Required	Description
`id`	string	Yes	Brand ID to fetch. Must be a valid identifier for an existing brand.

How to use in Postman

Set the path variable **id** (in the request URL path variables table) to the brand identifier you want to retrieve.

Expected responses (generic)

- **200 OK** — Brand found and returned.
- **400 Bad Request** — Invalid `id` format or validation error.
- **401 Unauthorized** — Missing/invalid authentication (if the API is protected).
- **404 Not Found** — No brand exists for the given `id`.
- **500 Internal Server Error** — Unexpected server error.

Example cURL

bash

```bash
curl --request GET "https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/<id>
  --header "Accept: application/json"
```

Example 200 response (placeholder)

json

```json
{
  "status": "success",
  "data": {
    "_id": "<id>",
    "name": "<brand_name>",
    "slug": "<brand_slug>",
    "image": "<image_url>",
    "createdAt": "<iso_datetime>",
    "updatedAt": "<iso_datetime>"
  }
}
```

Example error response (placeholder)

json

```json
{
  "status": "error",
  "message": "<error_message>
}
```

PATH VARIABLES

id

PUT update brand

https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/699cc33e2c556688103bba0d

Update Brand

Updates an existing brand by its id.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/:brandId`

Authentication

This endpoint requires a JWT token sent in a custom header:

```http
token: <JWT>
```

> Ensure your token is valid and has permission to update brands.

Path parameter

Name	Type	Required	Description
`brandId`	string	yes	The Brand id to update (MongoDB ObjectId). Example: `699cc33e2c556688103bba0d`

Request body

Content-Type: `application/json`

Schema:

```json
{
  "name": "string"
}
```

Example:

```json
```

```
{"name":"Runte, Larson and Wisoky-167"}
```

Responses

Success

- **200 OK** (typical): Brand updated and returned in the response body.

Example shape (varies by API implementation):

```json
{
  "_id": "699cc33e2c556688103bba0d",
  "name": "Runte, Larson and Wisoky-167",
  "updatedAt": "2026-03-01T12:34:56.789Z"
}
```

Errors

- **400 Bad Request**: Invalid payload (e.g., missing/invalid `name`) or invalid `brandId` format.
- **401 Unauthorized**: Missing/invalid `token` header.
- **403 Forbidden**: Authenticated but not allowed to update brands.
- **404 Not Found**: Brand id not found.
- **500 Internal Server Error**: Unexpected server error.

Notes

- `https://ecommerce-backend-production-8996.up.railway.app` is expected to be defined in the active environment named **`baseURL`**.
- If you get a DNS/connection error, verify the environment's `baseURL` value and that the correct environment is selected.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```json
```

```
{
    "name":"Runte, Larson and Wisoky-167"
}
```

DELETE delete brand

https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/699cc33e2c556688103bba0d

Delete Brand

Deletes a single brand by its ID.

Method: `DELETE`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/:brandId`

Authentication

This endpoint requires a JWT in the `token` header.

Header

```http
token: <JWT>
```

Path parameters

Param	Type	Required	Description
`brandId`	string (MongoDB ObjectId)	yes	The brand identifier to delete.

Environment / variables

Variable	Required	Description
`baseURL`	yes	Base URL for the API (e.g. `http://localhost:3000`).

Expected responses

Success

- **200 OK** (common): Brand deleted successfully (response body varies by API implementation).
- **204 No Content** (possible): Brand deleted, no response body.

Common errors

- **400 Bad Request**: Invalid `brandId` format.
- **401 Unauthorized**: Missing/invalid/expired JWT in `token` header.
- **403 Forbidden**: Authenticated but not allowed to delete brands.
- **404 Not Found**: No brand found for the given `brandId`.
- **500 Internal Server Error**: Unexpected server error.

Example cURL

```bash
curl --request DELETE "https://ecommerce-backend-production-8996.up.railway.app/api/v1/brands/
  --header "token: <JWT>"
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

---

coupon

POST createcoupon

https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons

Create Coupon

Creates a new coupon in the system.

Endpoint

`POST https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons`

Authentication

Send a token in the request headers:

```http
token: {{token}}
```

> Make sure `https://ecommerce-backend-production-8996.up.railway.app` points to your API host.

Request Body (JSON)

Field	Type	Required	Description
`discount`	number (integer)	Yes	Discount percentage/value for the coupon.
`code`	string	Yes	Unique coupon code.
`expires`	string	Yes	Expiration date (format depends on API, e.g. `DD-MM-YYYY`).

Example Request Body

```json
{
  "discount": 30,
  "code": "S012F5",
  "expires": "11-10-2027"
}
```

Typical Responses

- **201 Created** — Coupon created successfully.
- **400 Bad Request** — Validation error / missing or invalid fields.
- **401 Unauthorized** — Missing/invalid `token` header.
- **409 Conflict** — Coupon code already exists.
- **500 Internal Server Error** — Unexpected server error.

HEADERS

token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```json
{
    "discount":30,
    "code":"S012F5",
    "expires":"11-10-2027"

}
```

GET getcoupons

https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons

Purpose

Retrieves the list of coupons available in the system.

Endpoint

`GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons`

Authentication

This endpoint requires a token to be sent in the request headers:

```http
token: {{token}}
```

- The current request is configured with a `token` header.
- Store the token value in a variable (environment/collection) rather than hardcoding it.

Variables

- `https://ecommerce-backend-production-8996.up.railway.app`: Base URL for the API (set in the active environment).

Expected response (high level)

A JSON response containing coupon records.

Typically you can expect:

- A top-level JSON object
- A list/array of coupons (each coupon usually includes identifiers and coupon fields such as code/discount/expiry/status, depending on your API implementation)

Common status codes

- `200 OK` — Coupons returned successfully.
- `401 Unauthorized` / `403 Forbidden` — Missing/invalid token or insufficient permissions.
- `500 Internal Server Error` — Unexpected server error.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

PUT updatecoupon

https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons/699dd6cbea4a82568981b837

Update Coupon (PUT)

Updates an existing coupon's fields (commonly used to update the coupon **code**).

Authentication

This endpoint requires an admin/authorized token sent in the request headers:

```http
token: <JWT>
```

- In this request, the `token` header is already present.
- If the token is missing/invalid/expired, the API should return an authentication/authorization error (commonly `401` or `403`, depending on server implementation).

Request URL

```http
```

```
PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons/:couponId
```

Path parameters

Param	Type	Required	Description
`couponId`	string (MongoDB ObjectId)	yes	The coupon identifier to update.

Example (as currently configured):

```http
PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons/699dd6cbea4a825689
```

Request body

Content-Type: `application/json`

Schema

Field	Type	Required	Description
`code`	string	yes*	The coupon code value.

> *Required when you want to update the coupon code. (Other updatable fields may exist on the API, but this request currently updates `code`.)

Example

```json
{
  "code": "kklm234"
}
```

Responses

> Exact response body depends on the API implementation.

Success

- `200 OK` (common): coupon updated and returned in the response body, or a success message.
- `204 No Content` (possible): coupon updated with an empty response body.

Errors (common)

- `400 Bad Request` — validation error (e.g., missing/invalid `code`).
- `401 Unauthorized` — missing/invalid token.
- `403 Forbidden` — token valid but not allowed to update coupons.
- `404 Not Found` — coupon with `couponId` not found.

- `404 Not Found` — coupon with `couponId` not found.
- `409 Conflict` — duplicate coupon code (if code must be unique).
- `500 Internal Server Error` — unexpected server error.

Notes / Variables

- `https://ecommerce-backend-production-8996.up.railway.app` is a variable resolved from your active environment (currently: **baseURL**). Set it to your API host, for example:
  - `http://localhost:3000`
  - `https://api.yourdomain.com`
- The request uses a custom `token` header (not the standard `Authorization: Bearer ...`). Ensure you paste the correct JWT into the `token` header value.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```json
{
    "code":"kklm234"
}
```

DELETE deletecoupon

https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons/699dd6cbea4a82568981b837

Delete coupon

Deletes a single coupon by its id.

Method: `DELETE`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/coupons/:couponId`

Base URL

This request uses the environment variable `https://ecommerce-backend-production-8996.up.railway.app`.

Authentication

Requires a JWT passed in the `token` header.

```http
token: <JWT>
```

Path parameter

- `couponId` *(string, required)* — The coupon identifier.

> In this saved request, `couponId` is currently set to: `699dd6cbea4a82568981b837`.

Expected responses

- **200 OK** or **204 No Content** — Coupon deleted successfully.
- **401 Unauthorized** / **403 Forbidden** — Missing/invalid token or insufficient permissions.
- **404 Not Found** — Coupon id does not exist.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

product

POST add product

https://ecommerce-backend-production-8996.up.railway.app/api/v1/products

Purpose

Create a new product in the e-commerce system.

- **Method:** `POST`
- **URL:** `https://ecommerce-backend-production-8996.up.railway.app/api/v1/products`
- **Body type:** `multipart/form-data`

Authentication / Authorization

This endpoint requires an authenticated **admin** (or a role permitted to create products).

Required headers

Header	Required	Notes
`token`	Yes	JWT access token. Example: `token:`

> Note: If your API expects `Authorization: Bearer` instead, update your API/client accordingly. In this collection the header used is `token`.

Request body (form-data)

Send as **multipart/form-data**.

Field	Required	Type	Notes
`title`	Yes	string	Product title. Avoid wrapping in extra quotes.
`description`	Yes	string	Free text; can include new lines (`\n`).
`price`	Yes	number	Base price (e.g., `200`).
`stock`	Yes	number	Available quantity (integer).
`sold`	No*	number	Sold count. Often managed by the system; include only if your backend allows it.
`category`	Yes	string (ObjectId)	Category id (e.g., `69860316e610777971d4106f`).
`subCategory`	No/Yes**	string (ObjectId)	Subcategory id. Required if your backend enforces a category→subcategory

* **`sold`** is commonly server-calculated; confirm your backend rules.

** Some implementations require `subCategory` depending on schema/validation.

Example request

Headers

```http
```

```
http

token: {{token}}
```

form-data

```text
title: HTMLs and TAGs
description: matrial\npolyster Blend\nColor Name\tMulticolo
price: 200
stock: 254
sold: 100
category: 69860316e610777971d4106f
subCategory: 69877f4f902056165e7903af
brand: 6988ac47b186dcae2bf8059b
```

Example success response (201)

```json
{
  "message": "success",
  "product": {
    "_id": "<productId>",
    "title": "HTMLs and TAGs",
    "slug": "htmls-and-tags",
    "price": 200,
    "stock": 254,
    "sold": 100,
    "category": "<categoryId>",
    "subCategory": "<subCategoryId>",
```

Common errors

- **400 Bad Request**: Validation failed (missing required fields, invalid ObjectId format, wrong data types).
- **401 Unauthorized**: Missing/invalid `token`.
- **403 Forbidden**: Token is valid but user lacks permission (not admin / insufficient role).
- **409 Conflict** (if implemented): Duplicate product or slug conflict.
- **413 Payload Too Large** (if images/files are included and exceed limits).

Tips

- For numeric fields (`price`, `stock`, `sold`), send plain numbers (no quotes) when possible.
- If you see titles/slugs with extra quotes in the response, remove extra quotes from the `title` value in form-data.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoia2VybyBnZXJnZXMiLCJlbWFpbCI6ImtlcmonZXJnZXNAZ21haWwuY29tIiwiaWQiOiI2OWE0NmUxNTFhNjQwZDdmYTZiMmE0MmEiLCJyb2xlIjoiYWRtaW4iLCJpYXQiOjE3NzIzODM3ODI9.7Hw-h0ZOhoeQP8AmdGn03VFYD6miK3gYWg5z7Q7yfOg

Body formdata

title	"HTMLs and TAGs"
description	matrial\npolyster Blend\nColor Name\tMulticolo
price	200
stock	254
sold	100
category	69860316e610777971d4106f
subCategory	69877f4f902056165e7903af
brand	6988ac47b186dcae2bf8059b
imageCover	
images	

GET get products

https://ecommerce-backend-production-8996.up.railway.app/api/v1/products

Get products

Retrieves a list of products from the e-commerce API.

Purpose

- Fetch available products for listing/browsing in the client (e.g., storefront, admin dashboard).
- Supports server-side filtering/pagination **if** the API implements query parameters (not defined in this request).

Base URL / Environment variables

This request uses the environment variable `https://ecommerce-backend-production-8996.up.railway.app` as the API host:

text

```
https://ecommerce-backend-production-8996.up.railway.app/api/v1/products
```

Make sure your active environment defines `baseURL` (for example: `http://localhost:3000` or `https://api.yourdomain.com`).

Authentication

This endpoint expects a JWT token to be sent in a header named `token`:

```http
token: <JWT>
```

> If your API also supports the standard `Authorization: Bearer` header, you can switch to it—this request is currently configured to use the custom `token` header.

Expected responses

Common responses you may see:

- **200 OK** — Returns the list of products.
- **401 Unauthorized** — Missing/invalid token.
- **403 Forbidden** — Token is valid but lacks permission.
- **500 Internal Server Error** — Unexpected server-side error.

Example cURL

```bash
curl --request GET "https://ecommerce-backend-production-8996.up.railway.app/api/v1/products"
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiYmVzaG95IG1vZmllZCIsIm VtYWIsIjoiYmVzaG95MTIxQGdtYWIsLmNvbSIsImlkIjoiNjk5OWU2OWU2M2YxND AyMDJmOWM2YWI1IiwiaWF0IjoxNzcxNjk1MzIwfQ.YmMQEyc6KQbyLVF8wBbLX wQOA0QnvVk5fJrnWOgILGk

GET get product

https://ecommerce-backend-production-8996.up.railway.app/api/v1/products/699dd5f6ea4a82568981b82e

Get product by ID

Retrieves a single product resource by its unique identifier.

Endpoint

`GET https://ecommerce-backend-production-8996.up.railway.app/api/v1/products/:productId`

> Note: The currently saved URL in this request includes a concrete product id. You can replace it with any valid `:productId`.

Authentication

This endpoint requires an authenticated request.

This request is currently configured to send a JWT in a custom header:

```
Plain Text

token: {{token}}
```

- **Header name:** `token`
- **Value:** JWT string

> If your API also supports `Authorization: Bearer`, you can standardize on that, but **this request is documented to use the `token` header** as currently configured.

Variables

- `https://ecommerce-backend-production-8996.up.railway.app` — Base URL for the API server (for example: `http://localhost:3000` or `https://api.yourdomain.com`).

Recommended (optional) variable to avoid pasting tokens:

- `{{token}}` — JWT for an authenticated user.

Path parameters

- `productId` (string, required) — The product's unique id.

Example:

- `699dd5f6ea4a82568981b82e`

Success response (shape)

Typical **200 OK** response is a JSON object containing the product.

Example shape (fields may vary by your API implementation):

```json
{
  "message": "success",
  "data": {
    "_id": "699dd5f6ea4a82568981b82e",
    "title": "Product title",
    "description": "Product description",
    "price": 123,
    "quantity": 10,
    "images": ["https://..."],
    "category": {
      "_id": "...",
```

Common errors

- **400 Bad Request** — Invalid `productId` format.
- **401 Unauthorized** — Missing/expired/invalid JWT in the `token` header.
- **403 Forbidden** — Authenticated but not permitted to view the product.
- **404 Not Found** — No product exists for the given `productId`.
- **500 Internal Server Error** — Unexpected server-side failure.

Tips

- If you're testing multiple products, consider parameterizing the id:
  - Change the URL to `https://ecommerce-backend-production-8996.up.railway.app/api/v1/products/{{productId}}`
  - Store `productId` as a variable in an environment/collection.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiYmVzaG95IG1vZmllZCIsImVtYWlsIjoiYmVzaG95MTIxQGdtYWlsLmNvbSIsImlkIjoiNjk5OWU2OWU2M2YxNDAyMDJmOWM2YWI1IiwiaWF0IjoxNzcxNjk1MzIwfQ.YmMQEyc6KQbyLVF8wBbLXwQOA0QnvVk5fJrnWOgILGk

PUT update product

https://ecommerce-backend-production-8996.up.railway.app/api/v1/products/69933c231d7a2d4ed908021f

Update Product

Updates an existing product's data (e.g., title) by product ID.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app}/api/v1/products/:productId`

> **Note:** The currently saved URL appears to include an extra `}` after `https://ecommerce-backend-production-8996.up.railway.app` (i.e., `https://ecommerce-backend-production-8996.up.railway.app}/...`). This documentation calls it out, but the request itself is **not changed**.

Authentication (Required)

This endpoint requires an admin-auth token passed via a request header:

- Header: `token:`

Example:

```http
token: {{token}}
```

Path parameter

- `productId` (string) — The product's unique identifier.

Example (current request):

- `699dd5f6ea4a82568981b82e`

Request body

Send JSON with the fields you want to update.

Schema (JSON)

```json
{
  "title": "string"
}
```

Example

```json
{
```

```
  "title": "player and gamer stream"
}
```

Responses

Success

- Typically `200 OK` (or `204 No Content` depending on implementation)
- Returns the updated product object or a success message.

Errors

- `400 Bad Request` — Invalid payload / validation error.
- `401 Unauthorized` / `403 Forbidden` — Missing/invalid token or insufficient permissions.
- `404 Not Found` — Product ID not found.
- `500 Internal Server Error` — Unexpected server error.

Notes

- `https://ecommerce-backend-production-8996.up.railway.app` is resolved from the active environment (currently named **baseURL**). Ensure it points to your API host, e.g. `http://localhost:3000` or your deployed domain.
- If you change the token frequently, consider storing it as a variable (e.g., `{{token}}`) rather than hard-coding it in the request header.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiYmVzaG95IG1vZmlIZCIsImVtYWlsIjoiYmVzaG95MTIxQGdtYWlsLmNvbSIsImlkIjoiNjk5OWU2OWU2M2YxNDAyMDJmOWM2YWI1IiwiaWF0IjoxNzcxNjk1MzIwfQ.YmMQEyc6KQbyLVF8wBbLXwQOA0QnvVk5fJrnWOgILGk

Body raw (json)

```
  json

{
    "title":"player and gamer stream"
}
```

DELETE delete product

https://ecommerce-backend-production-8996.up.railway.app/api/v1/products/699dd5f6ea4a82568981b82e

Delete Product

Deletes a product by **ID**.

Purpose

Use this endpoint to permanently remove an existing product resource from the system.

Authorization

This request requires a valid JWT in the request headers:

```http
token: <jwt>
```

> Note: The header name for auth in this API is `token` (not `Authorization`). Make sure the token is for a user that has permission to delete products (commonly an admin role).

Required variables

This request uses the following Postman variable:

- `https://ecommerce-backend-production-8996.up.railway.app` — Base URL of the API (set in the active environment).

Path parameter

- `productId` — The product identifier to delete.

In this request, the ID is currently hard-coded in the URL:

- `699dd5f6ea4a82568981b82e`

Example request

```http
DELETE https://ecommerce-backend-production-8996.up.railway.app}/api/v1/products/699dd5f6ea4a8
token: <jwt>
```

Expected responses

Success

- **200 OK** (commonly returns a confirmation message or the deleted product)

- Example (varies by implementation):

```json
{
  "message": "Product deleted successfully"
}
```

- Some APIs may return **204 No Content** with an empty body.

Common errors

- **400 Bad Request** — Invalid/malformed product id.
- **401 Unauthorized** — Missing/invalid/expired `token`.
- **403 Forbidden** — Authenticated but not allowed to delete products.
- **404 Not Found** — No product exists for the provided id.
- **500 Internal Server Error** — Unexpected server-side failure.

Notes

- **Important:** The current URL contains an extra closing brace `}` right after `https://ecommerce-backend-production-8996.up.railway.app`:
  - Current: `https://ecommerce-backend-production-8996.up.railway.app}/api/v1/products/...`
  - Expected: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/products/...`

This can cause a malformed URL. Fix it when you're ready, but **this request has not been modified** as part of this documentation update.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

user

authentication

POST signup

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/signup

Signup (Create User)

Create a new user account.

Base URL

This request uses the environment variable:

- `https://ecommerce-backend-production-8996.up.railway.app` (from the **baseURL** environment)

Full endpoint:

- `POST https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/signup`

Authentication

- **None** (no auth required for signup)

Required headers

- `Content-Type: application/json`
- `Accept: application/json` *(recommended)*

Request body (JSON)

Field	Type	Required	Notes
`name`	`string`	Yes	User's display name. Typically a non-empty string.
`email`	`string`	Yes	Must be a valid email format and usually unique.
`password`	`string`	Yes	Must meet password validation rules (see **Validations** below).
`role`	`string`	Optional/Depends	User role. If omitted, API may default to a standard role (implementation-specific). See **Role values** below.

Example request payload

```json
{
  "name": "kero gerges",
  "email": "kerogerges@gmail.com",
  "password": "kero11",
  "role": "admin"
}
```

Example responses (documentation only)

201 Created (Success)

```json
{
  "status": "success",
  "data": {
    "user": {
      "_id": "64f0c2b8c0a1b2c3d4e5f678",
      "name": "kero gerges",
      "email": "kerogerges@gmail.com",
      "role": "admin"
    },
    "token": "<jwt_token>"
  }
```

400 Bad Request (Validation error)

```json
{
  "status": "fail",
  "message": "Validation error",
  "errors": [
    {
      "field": "email",
      "message": "Email is invalid"
    }
  ]
}
```

409 Conflict (Email already exists)

```json
{
```

```
{
  "status": "fail",


  "message": "Email already in use"
}
```

Notes

Validations

Exact rules depend on the backend implementation, but commonly:

- `name` : required, trimmed, minimum length.
- `email` : required, valid email format, must be unique.
- `password` : required, minimum length and/or complexity rules.
- `role` : must be one of the allowed role values.

Role values

Common patterns are role enums such as:

- `user`
- `admin`

If your API enforces a strict enum, sending an unsupported value should return a **400** validation error.

Body raw (json)

```
json

{
    "name":"kero gerges",
    "email":"kerogerges@gmail.com",
    "password":"kero11",
    "role":"admin"

}
```

POST signin

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/signin

Sign in

Purpose

Authenticate an existing user with email and password and obtain an access token (and user info) for subsequent authorized requests.

Endpoint

- **Method:** `POST`
- **URL:** `https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/signin`

Required variables

- `https://ecommerce-backend-production-8996.up.railway.app` — Base URL of the API (set in the active environment **baseURL**).

Headers

- `Content-Type: application/json`

Request body

Content-Type: `application/json`

Schema

```json
{
  "email": "string (required)",
  "password": "string (required)"
}
```

Example

```json
{
  "email": "<user-email>",
  "password": "<user-password>"
}
```

Expected responses

`200 OK`

Successful authentication.

- Typically returns a JSON payload that includes an auth token and user info, for example:
  - `token` (string)

- `token` (string)
- `user` (object)

`400 Bad Request`

Validation error (e.g., missing/invalid `email` or `password`).

`401 Unauthorized`

Invalid credentials (email/password incorrect) or user is not allowed to sign in.

Body raw (json)

```
json

{
    "email":"kerogerges@gmail.com",
    "password":"kero11"

}
```

POST add user

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users

Create user

Creates a new user account.

Endpoint

`POST https://ecommerce-backend-production-8996.up.railway.app/api/v1/users`

Authentication

Send a JWT in the `token` header:

```
http

token: {{token}}
```

> Note: This request currently uses a header named `token` (not `Authorization`).

Environment / Variables

- `baseURL` (required): Base URL of the API (for example `https://api.example.com`).

Request body (JSON)

Send `Content-Type: application/json` with the following fields:

Field	Type	Required	Notes
`name`	string	yes	User's display name.
`email`	string	yes	Must be a valid email address and typically must be unique.
`password`	string	yes	Password to set for the user (usually must meet complexity rules).

Example request

```json
{
  "name": "beshoy mofied",
  "email": "beshoy121@gmail.com",
  "password": "Besho1234"
}
```

Expected responses

201 Created (success)

Typical response shape (example):

```json
{
  "message": "User created successfully",
  "data": {
    "_id": "<userId>",
    "name": "beshoy mofied",
    "email": "beshoy121@gmail.com"
  }
}
```

400 Bad Request (validation error)

Returned when required fields are missing/invalid (for example invalid email, weak password).

Example shape:

```json
{
  "message": "Validation error",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email"
    }
  ]
}
```

401 Unauthorized / 403 Forbidden (auth)

Returned when the `token` header is missing/invalid/expired, or the caller does not have permission to create users.

Example shape:

```json
{
  "message": "Unauthorized"
}
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```json
{
    "name":"beshoy mofied",
    "email":"beshoy121@gmail.com",
    "password":"Besho1234"

}
```

GET get users

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users

GET `/api/v1/users`

Returns a list of users from the API.

Base URL / variable usage

This request uses the Postman variable `https://ecommerce-backend-production-8996.up.railway.app`:

- Full URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/users`

Make sure `baseURL` is set in your active environment (currently: `baseURL`).

Required headers

Header	Required	Description
`token`	Yes	API credential (Supabase **Service Role** key). Keep this secret; do not expose it in client apps.

Request details

- **Method:** `GET`
- **Body:** none
- **Query params:** none

Success response (example shape)

Status: `200 OK`

Typical response is an array of user objects (exact fields depend on your API implementation), for example:

```json
[
  {
    "id": "<user-id>",
    "email": "user@example.com",
    "name": "<optional>",
    "createdAt": "2026-01-01T00:00:00.000Z"
  }
]
```

Common error responses

- **400 Bad Request** — Invalid request (rare for this endpoint since it has no body/params).
- **401 Unauthorized** — Missing/invalid `token` header.
- **403 Forbidden** — `token` is valid but not permitted to read users.
- **500 Internal Server Error** — Unexpected server error.

Notes

- If you receive `401/403`, verify the `token` header is present and correct, and that the environment variable `https://ecommerce-backend-production-8996.up.railway.app` points to the intended server.

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

GET get user

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/699dd38bd0761e12a2905a87

Get user by id

Fetch a single user resource by its MongoDB `_id`.

Purpose

Use this endpoint to retrieve a user's profile/details when you already know the user id.

Request

- **Method:** `GET`
- **URL:** `https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/:userId`

Authentication

This request requires a JWT provided via the `token` header:

```http
token: <JWT>
```

> Make sure the `token` value is a valid JSON Web Token issued by your API (for example, from your sign-in endpoint).

Required variables

- `baseURL` — the API host (example: `http://localhost:3000`)

Path parameters

- `userId` *(string, required)* — the user's id.
  - Example from this request: `699dd38bd0761e12a2905a87`

Example cURL

```bash
curl --request GET "https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/699d
  --header "token: <JWT>"
```

Expected responses

200 OK

Returns the user object.

Sample shape:

```json
{
  "_id": "699dd38bd0761e12a2905a87",
  "name": "Jane Doe",
  "email": "jane@example.com",
  "role": "user",
  "createdAt": "2024-01-01T00:00:00.000Z",
  "updatedAt": "2024-01-01T00:00:00.000Z"
}
```

401 Unauthorized

Returned when the `token` header is missing/invalid/expired.

Sample shape:

```json
{
  "message": "Unauthorized"
}
```

404 Not Found

404 Not Found

Returned when no user exists for the given `userId`.

Sample shape:

```json
{
  "message": "User not found"
}
```

500 Internal Server Error

Returned on unexpected server-side errors.

Sample shape:

```json
{
  "message": "Internal server error"
}
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

PUT update user

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/699f41f7ec1cc2ac333e857f

Update user (PUT)

Updates an existing user record by user ID.

Endpoint

`PUT https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/:userId`

Authentication

This endpoint expects an API key/token in a request header:

```http
token: {{service_role_token}}
```

- Header name: `token`
- Value: Supabase Service Role API Key (keep this secret)

> Tip: store the token in an environment/collection variable (for example `service_role_token`) instead of hardcoding it.

Path parameters

Parameter	Type	Required	Description
`userId`	string	yes	The user id to update. Example: `699f41f7ec1cc2ac333e857f`

Request body

Content-Type: `application/json`

Schema (minimal):

```json
{
  "name": "string"
}
```

Example:

```json
{"name":"Beshoy Agaiby gamal"}
```

Responses

Typical responses you may see:

- **200 OK** — User updated successfully (response body returns the updated user or a success payload).
- **400 Bad Request** — Validation error (e.g., invalid JSON / missing or invalid fields).
- **401 Unauthorized / 403 Forbidden** — Missing/invalid token or insufficient permissions.
- **404 Not Found** — No user exists for the provided `userId`.
- **500 Internal Server Error** — Unexpected server-side error.

- **500 Internal Server Error** — Unexpected server-side error.

Notes

- Ensure `https://ecommerce-backend-production-8996.up.railway.app` is set in your active environment.
- Only include fields you want to update (if the API supports partial updates).

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```
{
    "name":"Beshoy Agaiby gamal"
}
```

DELETE delete user

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/699cf43673a2194b6065801e

Delete user

Deletes a user by **user id**.

This endpoint is typically used by an admin/service layer to remove a user record.

Authentication

This request uses a token header:

```
token: {{service_role_token}}
```

- The `token` value must be a valid API/service token with permission to delete users.
- **Do not hardcode secrets** in the request. Store them as an Environment/Collection variable instead.

Required variables

- `https://ecommerce-backend-production-8996.up.railway.app` — Base URL for the API (active environment: `baseURL`).
- `{{service_role_token}}` — Service/admin token used in the `token` header (recommended as a **secret** variable).

Path parameter

- `userId` — The user identifier to delete.

Recommended URL pattern:

```
Plain Text

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/{{userId}}
```

Expected responses

Common responses you may see:

- **200 OK / 204 No Content** — User deleted successfully.
- **400 Bad Request** — Invalid `userId` format.
- **401 Unauthorized / 403 Forbidden** — Missing/invalid token or insufficient permissions.
- **404 Not Found** — User does not exist.
- **429 Too Many Requests** — Rate limited.
- **500 Internal Server Error** — Unexpected server error.

Safe example (using variables)

Set these in your environment/collection variables:

- `baseURL = https://your-api-host.tld`
- `userId = 699cf43673a2194b6065801e`
- `service_role_token =`

Then call:

```
Plain Text

DELETE https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/{{userId}}
```

Header:

```
Plain Text

token: {{service_role_token}}
```

```
token: {{service_role_token}}
```

HEADERS

token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmlsZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

PATCH change user password

https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/699f41f7ec1cc2ac333e857f

Update a user's password

Updates the password for a specific user.

Method: `PATCH`
URL: `https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/:userId`

Authentication

This endpoint requires a Supabase **Service Role** key.

Provide it via the request header:

```http
token: {{SUPABASE_SERVICE_ROLE_KEY}}
```

> Keep this key secret. It has elevated privileges.

Headers

Header	Required	Description
`token`	Yes	Supabase Service Role API key used to authorize the operation.
`Content-Type`	Recommended	`application/json` for JSON bodies. (Postman typically sets this automatically when using raw JSON.)

Path parameters

Path parameters

Parameter	Required	Type	Description
`userId`	Yes	string	The user identifier to update. Example: `699f41f7ec1cc2ac333e857f`

Request body (JSON)

Send a JSON object in the request body.

Field	Required	Type	Description
`password`	Yes	string	The new password to set for the user.

Example body:

```json
{
  "password": "beshoy1234"
}
```

Example request

```http
PATCH https://ecommerce-backend-production-8996.up.railway.app/api/v1/users/699f41f7ec1cc2ac333
token: <SUPABASE_SERVICE_ROLE_KEY>
Content-Type: application/json

{
  "password": "beshoy1234"
}
```

Example success response

Exact response shape depends on your API implementation. Common patterns include:

200 OK

```json
```

```
{
  "message": "User updated successfully",
  "data": {
    "id": "699f41f7ec1cc2ac333e857f"
  }
}
```

Or **204 No Content** (no response body).

Common errors

Status	When it happens
`400 Bad Request`	Missing/invalid `password` field or invalid JSON.
`401 Unauthorized`	Missing `token` header or invalid/expired key.
`403 Forbidden`	Key is valid but lacks the required privileges (or server policy blocks the action).
`404 Not Found`	`userId` does not exist.
`409 Conflict`	Update rejected due to a conflicting state (if enforced by your backend).
`422 Unprocessable Entity`	Validation failed (e.g., password policy / minimum length).
`500 Internal Server Error`	Unexpected server-side error.

Notes / security

- Use HTTPS.
- Avoid logging the `token` or the new `password`.
- Consider enforcing password complexity and rate limiting on the server.

HEADERS

token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiQmVzaG8iLCJlbWFpbCI6ImJlc2hvQGdtYWlsLmNvbSIsImlkIjoiNjk5ZGNhMjUyZDg0MTk3ZThjNGY0ODY2IiwicmosZSI6ImFkbWluIiwiaWF0IjoxNzcxOTQ5NjgwfQ.ilmZ0CfrDQxFR6IfBx5ywPyW5qxyYr-dQTMNc3FdImQ

Body raw (json)

```
json
```

```
{
    "password":"beshoy1234"
}
```

logout

POST logout

https://ecommerce-backend-production-8996.up.railway.app/api/v1/logout

Logout

Logs the currently authenticated user out by invalidating the active session/token on the server.

Authentication

This endpoint requires an authenticated session. Use **one** of the following:

- **Cookie-based session**: send the session cookie (typically `Cookie: <session_name>=<value>`). Postman will usually manage cookies automatically if you have already signed in.
- **Bearer token**: send an access token header:

```http
Authorization: Bearer {{access_token}}
```

> Replace `{{access_token}}` with your token variable (environment/collection variable), or paste a real token for manual testing.

Required headers

- `Authorization: Bearer {{access_token}}` *(if using token auth)*
- `Cookie: ...` *(if using cookie/session auth)*
- `Accept: application/json` *(recommended)*

> No request body is required.

Base URL variable

The request URL uses the environment variable `https://ecommerce-backend-production-8996.up.railway.app`:

- `https://ecommerce-backend-production-8996.up.railway.app` should point to your API host, for example: `https://api.yourdomain.com`

- Final endpoint:

```text
https://ecommerce-backend-production-8996.up.railway.app/api/v1/logout
```

Expected responses

- **200 OK** or **204 No Content** — Logout successful
- **401 Unauthorized** — Not authenticated / missing or invalid credentials (token/cookie)

cURL example

```bash
curl --request POST "https://ecommerce-backend-production-8996.up.railway.app/api/v1/logout" \
  --header "Authorization: Bearer {{access_token}}" \
  --header "Accept: application/json"
```